



Manual técnico de StorySphere



Eilyn Ortiz Guevara
Kebin Alejandro Manco Varela
Yerson Orejuela Arroyo

Servicio Nacional de Aprendizaje, Centro tecnológico del mobiliario
ADSO
04 de marzo de 2025

Tabla de contenido

Tabla de contenido	2
Información general del software.....	6
Versión y fecha de lanzamiento	6
Descripción general.....	7
Propósito	7
Público objetivo	7
Tecnologías usadas.....	8
<i>Php con laravel.....</i>	<i>8</i>
<i>MYSQL.....</i>	<i>8</i>
<i>React.js con Vite.....</i>	<i>9</i>
Casos de Uso	10
Administrador	10
Usuario Lector.....	11
Usuario Escritor	11
Usuario con Suscripción	11
Requisitos previos	12
Hardware:.....	12
Software	12
Dependencias en React con Vite	12
<i>Dependencias por defecto</i>	<i>12</i>
<i>Dependencias adicionales.....</i>	<i>13</i>
Dependencias del Backend en Laravel.....	14

<i>Dependencias por defecto</i>	<i>14</i>
<i>Dependencias adicionales.....</i>	<i>14</i>
Configuración inicial y variables de entorno.....	15
<i>Configuración General.....</i>	<i>15</i>
<i>Configuración de Base de Datos</i>	<i>16</i>
<i>Configuración de Sesión.....</i>	<i>16</i>
<i>Configuración de Caché y Colas</i>	<i>16</i>
<i>Configuración de Redis y Memcached.....</i>	<i>16</i>
<i>Configuración de Correo Electrónico.....</i>	<i>17</i>
<i>Configuración de Cloudinary (Almacenamiento de Imágenes)</i>	<i>17</i>
<i>Configuración del Frontend.....</i>	<i>17</i>
Arquitectura del sistema	18
Estructura general del software	18
<i>Frontend – React.js con Vite</i>	<i>18</i>
<i>Backend – Laravel</i>	<i>18</i>
<i>Base de datos – MYSQL.....</i>	<i>18</i>
<i>Comunicación entre módulos.....</i>	<i>19</i>
Diagramas UML.....	19
<i>Diagrama caso de uso</i>	<i>19</i>
<i>Diagrama de clases</i>	<i>20</i>
<i>Diagrama de relaciones</i>	<i>21</i>
<i>Diagrama de escenarios y roles</i>	<i>22</i>
Patrón de diseño utilizado	22

<i>Arquitectura cliente-servidor</i>	22
<i>Patrón Modelo-Vista-Controlador (MVC) en Laravel</i>	23
<i>Patrón de componentes en React</i>	23
Explicación de módulos y su interconexión	23
<i>Autenticación (Auth)</i>	23
<i>Usuarios (Users)</i>	24
<i>Historias (Stories)</i>	24
<i>Comentarios (Comments)</i>	25
<i>Interacciones (Likes y Ratings)</i>	26
<i>Seguidores (Followers)</i>	26
Desarrollo y estructura del código	28
Principales clases y funciones	28
<i>User.php (Usuario)</i>	28
<i>Story.php (Historia)</i>	28
<i>Comment.php (Comentario)</i>	29
<i>Like.php (Me gusta)</i>	29
<i>Rating.php (Calificación)</i>	29
<i>Category.php (Categoría)</i>	29
<i>Role.php (Rol de usuario)</i>	30
<i>Subscription.php (Suscripción)</i>	30
<i>Invoice.php (Factura)</i>	30
Base de datos	31
Modelo de datos y diagramas entidad-relación	31



<i>Modelo de datos.....</i>	<i>31</i>
<i>Modelo entidad - relación.....</i>	<i>32</i>
Tablas principales y relaciones	32
Procedimientos almacenados y consultas SQL relevantes	33
API y servicios	38
Documentación de endpoints (REST, GraphQL, SOAP, etc.)	38
<i>Rutas para autenticación.....</i>	<i>38</i>
<i>Rutas para usuarios</i>	<i>38</i>
<i>Historias de un usuario.....</i>	<i>38</i>
<i>Resetear contraseña</i>	<i>39</i>
<i>Rutas para historias</i>	<i>39</i>
<i>Rutas públicas</i>	<i>39</i>
<i>Rutas protegidas.....</i>	<i>40</i>
<i>Actualizar estado de una historia</i>	<i>40</i>
<i>Rutas para los comentarios</i>	<i>40</i>
<i>Rutas públicas</i>	<i>40</i>
<i>Rutas protegidas.....</i>	<i>41</i>
<i>Rutas para interacciones</i>	<i>41</i>
<i>Rutas para ratings.....</i>	<i>42</i>
<i>Rutas para suscripciones</i>	<i>42</i>
<i>Rutas para facturas.....</i>	<i>42</i>
Ejemplos de solicitudes y respuestas	43
Diccionario de variables	47



Users	47
Stories	49
Subscriptions	49
Statistic_stories.....	50
Roles	51
Likes	51
Invoices.....	52
Comments	53
Category_story	54
Role_user	54
Ratings	55
Personal_access_tokens	55
Webgrafía	56
Código fuente.....	56
Diagramas.....	56

Información general del software

Versión y fecha de lanzamiento

La versión Beta de la aplicación fue lanzada el 22 de abril, marcando una etapa clave en el desarrollo del proyecto. Esta versión permite a los usuarios probar las funcionalidades principales



y proporcionar retroalimentación de cómo actúa la página y sus usuarios ante una presencia real en el internet.

Descripción general

Este manual proporciona las instrucciones necesarias para la instalación y configuración del software Storysphere en un entorno adecuado. Incluye los requisitos previos, la descarga de archivos, la instalación de dependencias y la verificación del correcto funcionamiento del sistema.

El objetivo es garantizar una instalación fluida y sin errores, permitiendo que el usuario configure el software de manera óptima para su uso.

Propósito

El propósito de este manual es facilitar la instalación y puesta en marcha de Storysphere, asegurando que los usuarios sigan un proceso claro y estructurado. Este documento está dirigido a desarrolladores y administradores de sistemas, proporcionando los pasos detallados y las mejores prácticas para evitar problemas técnicos durante la instalación.

Público objetivo

Este manual está dirigido a desarrolladores, administradores de sistemas y cualquier persona encargada de la instalación y configuración de la página. Se asume que los usuarios tienen conocimientos básicos en desarrollo web, bases de datos y administración de servidores. Los principales perfiles que pueden beneficiarse de este manual incluyen:

- Desarrolladores Backend que necesiten instalar y configurar la API en Laravel.

- Desarrolladores Frontend que trabajen con React.js y requieran conectar la interfaz con el backend.
- Administradores de Bases de Datos encargados de gestionar SQL Server y su integración con la aplicación.
- Administradores de Sistemas que realicen la instalación en entornos de producción o servidores locales.

Tecnologías usadas

Php con laravel

PHP es un lenguaje de programación utilizado para el desarrollo web, permitiendo la integración con HTML y la ejecución de código en el servidor. Esto facilita la generación de contenido dinámico y la interacción con múltiples bases de datos.

Para este proyecto, se ha utilizado Laravel, un framework de PHP que ofrece una estructura organizada y herramientas avanzadas para el desarrollo rápido y eficiente de aplicaciones. Laravel ha sido empleado específicamente para la creación de una API, lo que facilita la comunicación entre el backend y el frontend, garantizando una arquitectura escalable y segura.

MYSQL

La base de datos utilizada en este proyecto es MYSQL Server, un sistema de gestión de bases de datos relacional, proporciona un alto rendimiento, seguridad avanzada y opciones de escalabilidad, asegurando la integridad de los datos y optimizando las consultas. Entre sus características clave están:

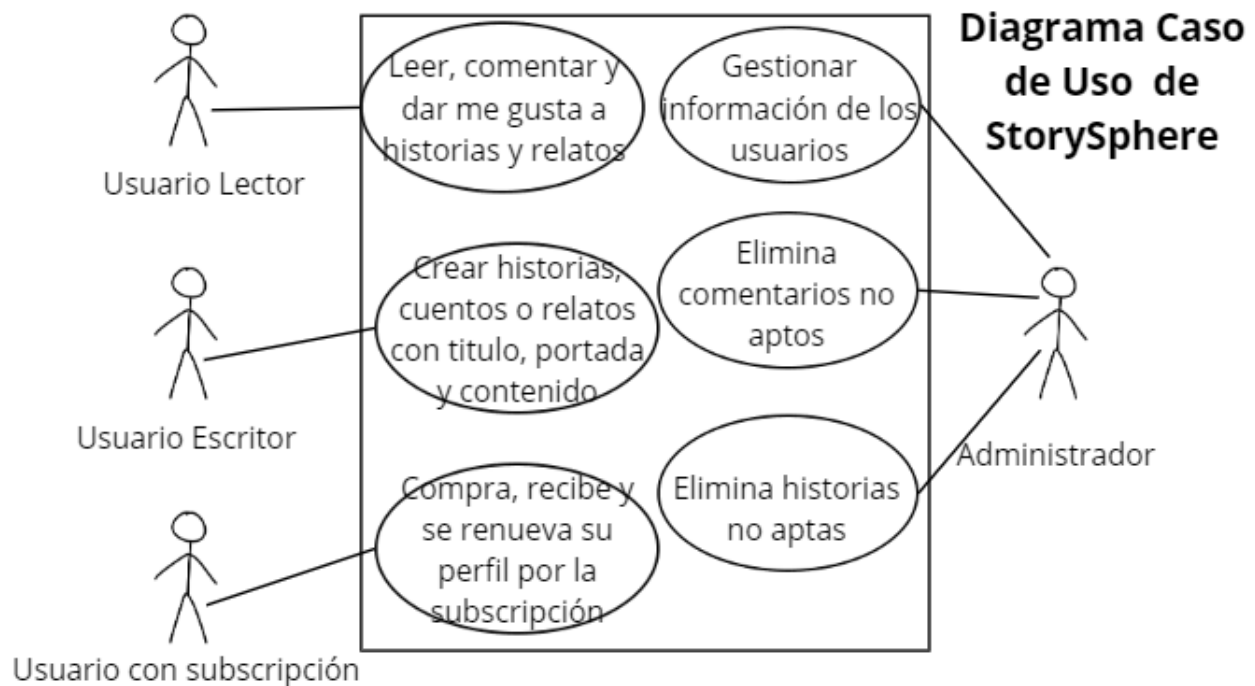
- Alta disponibilidad y replicación para entornos de producción.
- Optimización de consultas mediante índices y procedimientos almacenados.
- Seguridad avanzada, incluyendo autenticación y cifrado de datos.

React.js con Vite

Para el desarrollo del frontend, se ha utilizado React.js con Vite, una biblioteca de JavaScript que permite la creación de interfaces de usuario dinámicas y eficientes. Vite se encarga de optimizar la carga y compilación del código, mejorando la velocidad de desarrollo y la experiencia del usuario. Entre las ventajas de esta tecnología se encuentran:

- Renderizado eficiente gracias a la arquitectura basada en componentes.
- Desarrollo rápido con recarga en tiempo real y optimización del código.
- Integración fluida con estilos CSS dentro de JavaScript, facilitando la personalización de la interfaz.

Casos de Uso



Para el aplicativo StorySphere se han definido tres tipos de usuarios con roles específicos que interactúan con el sistema de diferentes maneras.

Administrador

El administrador tiene la responsabilidad de gestionar la plataforma y moderar el contenido generado por los usuarios. Sus funciones incluyen:

- Gestionar la información de los usuarios.
- Eliminar comentarios no aptos.
- Eliminar historias que no cumplan con las normas de la plataforma.

Usuario Lector

El usuario lector es aquel que interactúa con las historias publicadas. Sus principales acciones dentro del sistema son:

- Leer historias, cuentos o relatos.
- Comentar en las publicaciones.
- Dar "me gusta" a historias y relatos.

Usuario Escritor

El usuario escritor tiene la capacidad de generar contenido para la plataforma. Sus funciones incluyen:

- Crear historias, cuentos o relatos.
- Asignar título, portada y contenido a sus publicaciones.

Usuario con Suscripción

Este usuario tiene acceso a beneficios adicionales mediante un modelo de suscripción. Sus actividades incluyen:

- Comprar y renovar su suscripción.
- Recibir mejoras en su perfil como parte de la suscripción.

Con estos roles definidos, el sistema garantiza una experiencia organizada y fluida para todos los usuarios de StorySphere.



Requisitos previos

Hardware:

- Sistema Operativo: Windows 10 / Linux (Ubuntu 20.04 o superior) / macOS.
- Procesador: Intel Core i3 / AMD Ryzen 3 o superior.
- Memoria RAM: 2GB (se recomienda 8GB o más para un mejor rendimiento).
- Disco Duro: 10GB de espacio libre en el almacenamiento (preferiblemente SSD para mayor velocidad).
- Resolución de pantalla: 1366 x 768 píxeles (se recomienda Full HD o superior para mejor experiencia).
- Periféricos: Teclado y ratón (auriculares opcionales).

Software

- Privilegios de administrador para la instalación y configuración del sistema.
- React.js versión 16 o superior.
- Npm (Node Package Manager) como gestor de paquetes para React.js.
- Git para la gestión del código y control de versiones.
- Visual Studio Code o cualquier otro editor de código compatible con Vite.
- MySQL como motor de base de datos para la gestión de información.

Dependencias en React con Vite

Dependencias por defecto

Al crear el proyecto con Vite y se elige React, se incluyen estas dependencias:

- react: Biblioteca principal de React.
- react-dom: Permite la renderización de componentes React en el DOM.
- vite: Herramienta de desarrollo para una compilación rápida y eficiente.

Dependencias adicionales

Iconos y gráficos

- @fortawesome/fontawesome-svg-core, @fortawesome/free-brands-svg-icons, @fortawesome/free-solid-svg-icons, @fortawesome/react-fontawesome
- Conjunto de iconos de FontAwesome para React.
- lucide-react: Biblioteca de iconos moderna y ligera.

Animaciones y efectos visuales

- animate.css: Biblioteca de animaciones CSS predefinidas.
- framer-motion: Biblioteca para animaciones avanzadas en React.

Interacción con APIS y datos

- axios: Cliente HTTP para hacer peticiones a APIS.

Componente UI y formularios

- react-hook-form: Manejo de formularios de manera eficiente en React.
- react-toastify: Notificaciones emergentes en React.
- react-rating-stars-component: Componente para mostrar calificaciones con estrellas.

Navegación y estado de la aplicación

- react-hook-form: Manejo de formularios de manera eficiente en React.
- react-toastify: Notificaciones emergentes en React.
- react-rating-stars-component: Componente para mostrar calificaciones con estrellas.



Efectos especiales y partículas

- react-particles: Biblioteca para generar efectos de partículas.
- tsparticles, tsparticles-engine, tsparticles-slim: Extensiones de partículas personalizables.

Carruceles y desplazamiento

- swiper: Biblioteca para crear sliders y carruseles.

Dependencias del Backend en Laravel

Dependencias por defecto

- php: Lenguaje de programación principal del backend.
- laravel/framework: El núcleo del framework Laravel que proporciona todas las funcionalidades esenciales.
- laravel/tinker: Permite interactuar con la aplicación desde la línea de comandos mediante una consola interactiva.

Dependencias adicionales

Autenticación y seguridad

laravel/sanctum: Proporciona autenticación basada en tokens para API.

Gestión de archivos y almacenamiento

cloudinary/cloudinary_php: Permite la integración con Cloudinary para la gestión de imágenes y recursos multimedia.

Testing y desarrollo

- fakerphp/faker: Generador de datos ficticios para pruebas y desarrollo.
- phpunit/phpunit: Framework de pruebas unitarias para PHP.



- mockery/mockery: Biblioteca para la simulación de objetos en pruebas.
- nunomaduro/collision: Mejora la visualización de errores en la consola.

Optimización y herramientas de desarrollo

- laravel/pail: Herramienta para visualizar logs en tiempo real.
- laravel/pint: Herramienta de formateo de código para mantener estándares de codificación.
- laravel/sail: Configuración de entorno de desarrollo basado en Docker.

Scripts y automatización

Laravel incluye scripts que facilitan tareas como la configuración del entorno, la migración de bases de datos y la gestión de dependencias:

- @php artisan key:generate: Genera la clave de cifrado de la aplicación.
- @php artisan migrate --graceful --ansi: Ejecuta las migraciones de la base de datos.
- npx concurrently: Permite ejecutar múltiples procesos de desarrollo en paralelo (servidor, cola de tareas, logs, Vite).

Configuración inicial y variables de entorno

Configuración General

- APP_ENV=local: Define el entorno de la aplicación (local, producción, staging, etc.).
- APP_NAME=Laravel: Nombre de la aplicación.
- APP_KEY=...: Clave secreta utilizada para la encriptación y seguridad.
- APP_DEBUG=true: Indica si la aplicación está en modo depuración.
- APP_TIMEZONE=UTC: Zona horaria de la aplicación.
- APP_URL=http://localhost: URL base de la aplicación.



Configuración de Base de Datos

- DB_CONNECTION=mysql: Define el gestor de base de datos.
- DB_HOST=127.0.0.1: Dirección del servidor de la base de datos.
- DB_PORT=3306: Puerto por defecto de MySQL.
- DB_DATABASE=storysphere: Nombre de la base de datos.
- DB_USERNAME=root: Usuario de la base de datos.
- DB_PASSWORD=: Contraseña (vacía en desarrollo).

Configuración de Sesión

- SESSION_DRIVER=file: Define el método de almacenamiento de sesiones (archivo, base de datos, etc.).
- SESSION_LIFETIME=120: Duración de la sesión en minutos.

Configuración de Caché y Colas

- CACHE_STORE=database: Define la base de datos como almacenamiento de caché.
- QUEUE_CONNECTION=database: Usa la base de datos para manejar colas de trabajos en segundo plano.

Configuración de Redis y Memcached

- REDIS_CLIENT=phpredis: Define Redis como el cliente de caché.
- REDIS_HOST=127.0.0.1: Servidor Redis.
- REDIS_PORT=6379: Puerto Redis.
- MEMCACHED_HOST=127.0.0.1: Configuración de Memcached.



Configuración de Correo Electrónico

- MAIL_MAILER=smtp: Define el protocolo de envío de correos.
- MAIL_HOST=smtp.gmail.com: Servidor SMTP de Gmail.
- MAIL_PORT=587: Puerto SMTP con cifrado TLS.
- MAIL_USERNAME=storysphere.official@gmail.com: Usuario de correo.
- MAIL_PASSWORD=mjaodrhmcfcniop: Contraseña de la cuenta (debería usarse una variable segura en producción).
- MAIL_FROM_ADDRESS=storysphere.official@gmail.com: Dirección desde la que se envían los correos.
- MAIL_FROM_NAME="Storysphere": Nombre mostrado en los correos.

Configuración de Cloudinary (Almacenamiento de Imágenes)

- CLOUDINARY_URL=...: URL de conexión con Cloudinary.
- CLOUDINARY_CLOUD_NAME=dskr3jxir: Nombre del espacio en Cloudinary.
- CLOUDINARY_API_KEY=485247684388558: Clave de API.
- CLOUDINARY_API_SECRET=acHtkDhOdZmbyJAS_PSTL2eyyi8: Secreto de la API.

Configuración del Frontend

- FRONTEND_URL=http://localhost:3000: Define la URL del frontend que interactúa con el backend.

Arquitectura del sistema

Estructura general del software

El sistema está diseñado con una arquitectura cliente-servidor, donde el frontend, el backend y la base de datos trabajan de manera interconectada para proporcionar una experiencia fluida a los usuarios.

Frontend – React.js con Vite

- Se encarga de la interacción con el usuario, proporcionando una interfaz gráfica dinámica y responsiva.
- Utiliza React.js para la gestión de componentes reutilizables.
- Se apoya en Vite para una compilación rápida y eficiente.
- Usa bibliotecas como React Router para la navegación, Axios para consumir la API, y Framer Motion para animaciones.

Backend – Laravel

- Implementa una API RESTful en PHP con Laravel para manejar las solicitudes del frontend.
- Gestiona la autenticación de usuarios, validaciones y la lógica del negocio.
- Se encarga de procesar datos y devolver respuestas en formato JSON al frontend.

Base de datos – MYSQL

- Almacena y gestiona la información del sistema de forma estructurada.

- Utiliza MYSQL Server, un sistema de bases de datos relacionales (RDBMS) con alta escalabilidad.
- Laravel interactúa con la base de datos a través de Eloquent ORM, facilitando la manipulación de datos con PHP.

Comunicación entre módulos

- Frontend ↔ Backend: Se comunican a través de peticiones HTTP utilizando Axios en React y controladores en Laravel.
- Backend ↔ Base de datos: Laravel usa Eloquent ORM y consultas SQL para interactuar con SQL Server.

Diagramas UML

Diagrama caso de uso

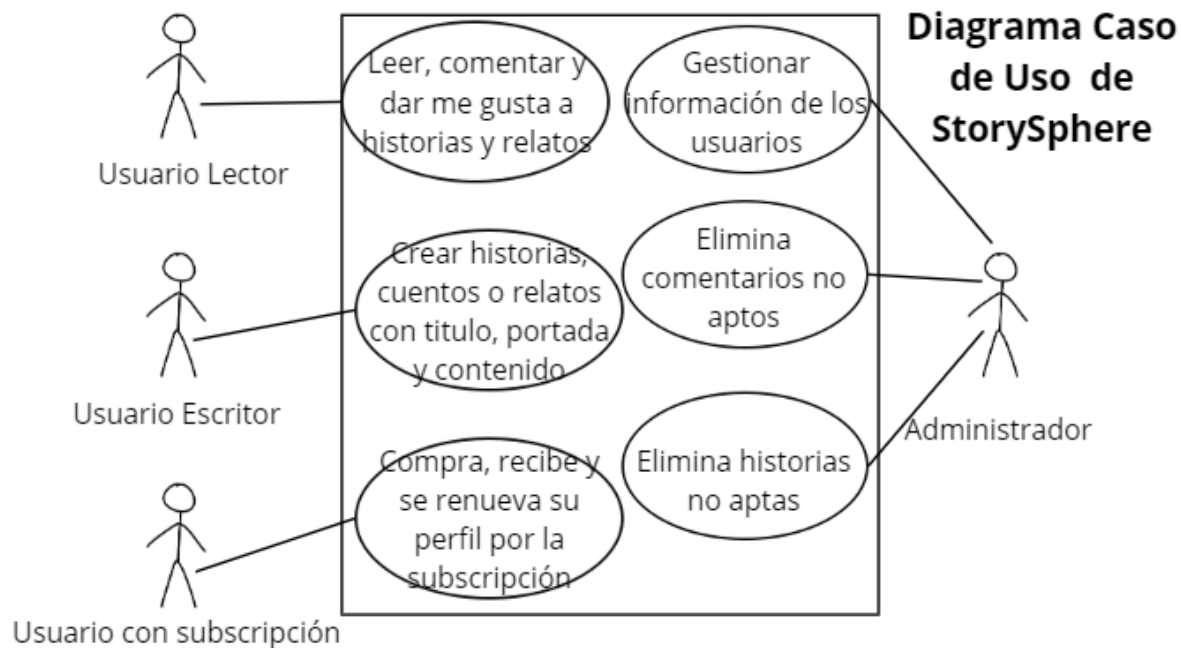


Diagrama de clases

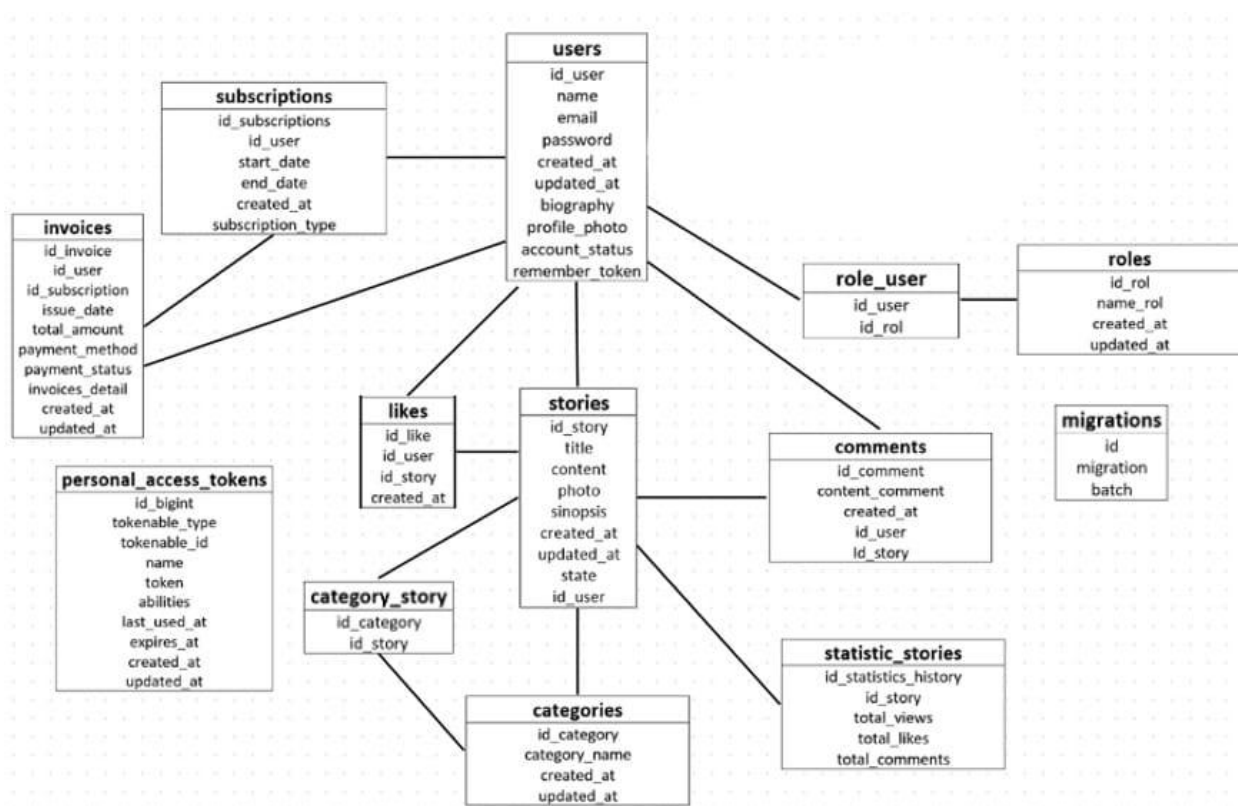


Diagrama de relaciones

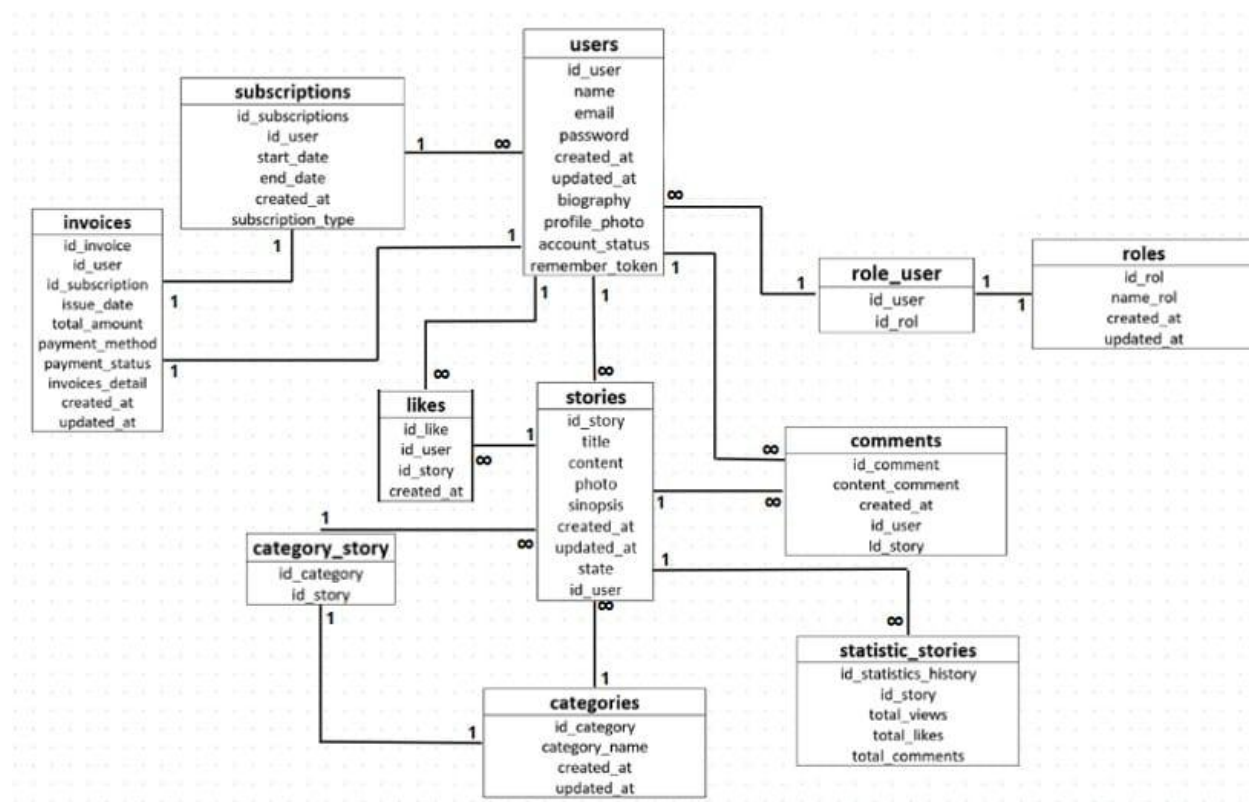
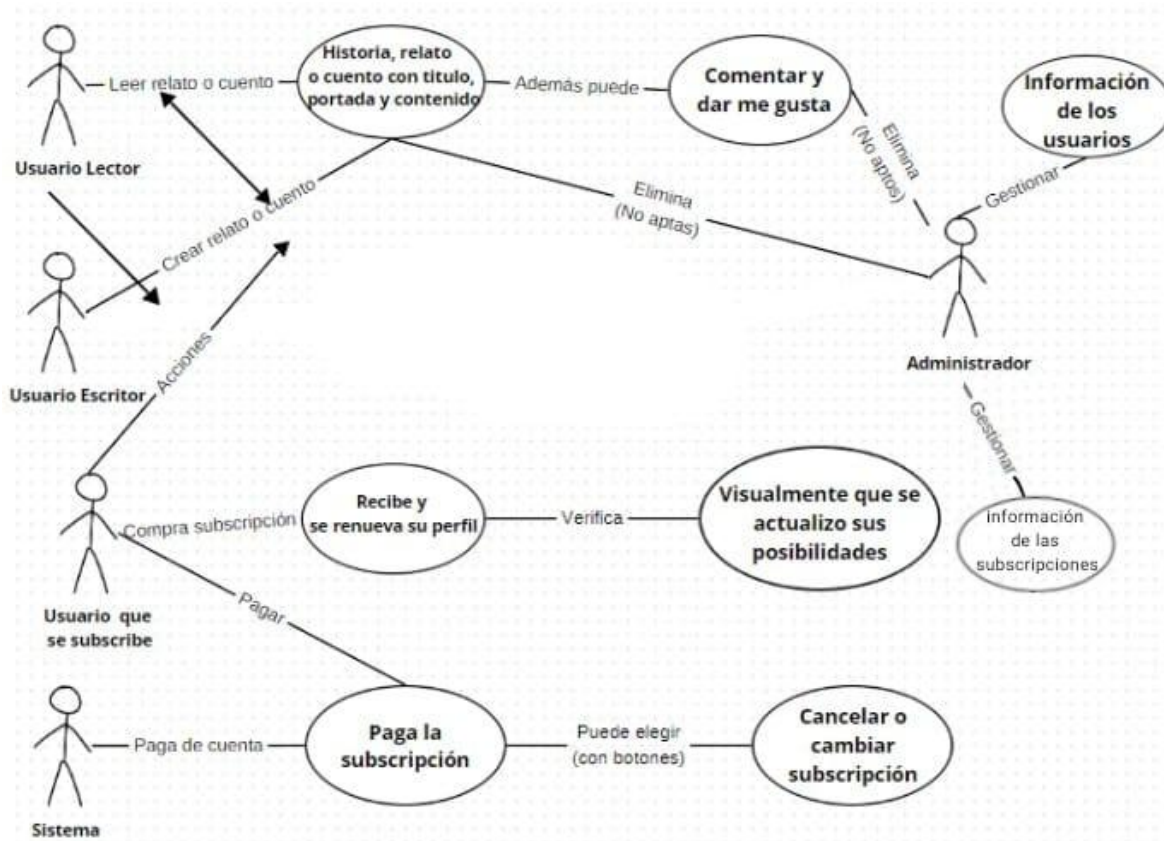


Diagrama de escenarios y roles



Patrón de diseño utilizado

Arquitectura cliente-servidor

- El cliente (React con Vite) envía solicitudes y recibe respuestas del servidor.
- El servidor (Laravel) procesa las solicitudes y gestiona los datos con la base de datos (SQL Server).

Este patrón separa la interfaz de usuario (frontend) de la lógica del negocio (backend), facilitando la escalabilidad y mantenimiento del proyecto.

Patrón Modelo-Vista-Controlador (MVC) en Laravel

Laravel sigue el patrón MVC, que organiza la aplicación en tres capas:

- **Modelo (Model):** Representa la estructura de los datos y gestiona la lógica de la base de datos con Eloquent ORM.
- **Vista (View):** Se encarga de mostrar la información (aunque en tu caso, React gestiona la vista).
- **Controlador (Controller):** Maneja la lógica de la aplicación y responde a las solicitudes del usuario.

Patrón de componentes en React

React sigue el patrón de componentes, donde la interfaz de usuario se divide en partes reutilizables y modulares.

- Cada componente maneja su propio estado y lógica.
- Los componentes pueden comunicarse entre sí mediante props y eventos.
- Esto facilita la reutilización y el mantenimiento del código.

Explicación de módulos y su interconexión

Autenticación (Auth)

```
Route::controller(AuthController::class)->group(function () {
```

```
    Route::post('/register', 'register'); // Registrar nuevo usuario
```

```
    Route::post('/login', 'login'); // Loguear usuario
```

```
    Route::middleware(['auth:sanctum', 'token.expiration'])->post('/logout', 'logout'); // Cerrar sesión
```

```
});
```

Interconexión:

- Frontend (React con Axios) envía credenciales (email, password) a Laravel.
- Laravel genera un token con Sanctum y lo envía al frontend.
- Las rutas protegidas requieren este token en cada petición.

Usuarios (Users)

```
Route::prefix('users')->middleware(['auth:sanctum', 'token.expiration'])->controller(UserController::class)->group(function () {

    Route::get('/', 'index'); // Obtener todos los usuarios

    Route::get('{id}', 'show'); // Obtener un usuario específico

    Route::delete('{id}', 'deleteAccount'); // Eliminar cuenta

    Route::put('{id}/role', 'updateRole'); // Actualizar rol de usuario

});
```

Interconexión:

- El frontend obtiene información de los usuarios y permite que administradores cambien roles.
- Laravel consulta la base de datos (users table).

Historias (Stories)

```
Route::prefix('stories')->group(function () {

    Route::get('/', [StoryController::class, 'index']); // Obtener todas las historias

    Route::get('{id}', [StoryController::class, 'show']); // Obtener una historia específica
```




Route::get('/{story}/comments', [CommentController::class, 'index']); // Comentarios de una historia

```
Route::middleware(['auth:sanctum', 'token.expiration'])->group(function () {
    Route::post('/', [StoryController::class, 'store']); // Crear historia
    Route::put('/{story}', [StoryController::class, 'update']); // Actualizar historia
    Route::delete('/{story}', [StoryController::class, 'destroy']); // Eliminar historia
});
});
```

Interconexión:

- Los escritores pueden crear, editar y eliminar historias si están autenticados.
- Los lectores pueden ver historias sin autenticación.
- Laravel maneja relaciones entre historias y usuarios en la base de datos.

Comentarios (Comments)

```
Route::prefix('comments')->group(function(){
    Route::get('/{comment}', [CommentController::class, 'show']); // Obtener un comentario
    Route::middleware(['auth:sanctum', 'token.expiration'])->group(function(){
        Route::post('/{storyId}', [CommentController::class, 'store']); // Agregar comentario
        Route::put('/{comment}', [CommentController::class, 'update']); // Editar comentario
        Route::delete('/{comment}', [CommentController::class, 'destroy']); // Borrar comentario
    });
});
```

Interconexión:

- Laravel maneja comentarios relacionados con historias y usuarios autenticados.
- El frontend usa Axios para enviar los comentarios a la y RaAPI.

Interacciones (Likes y Ratings)

```
Route::prefix('interactions')->middleware(['auth:sanctum', 'token.expiration'])->group(function ()
{
    Route::put('/{story}', [LikeController::class, 'toggleInteraction']); // Like/Unlike historia
});
```

```
Route::prefix('ratings')->middleware(['auth:sanctum', 'token.expiration'])->group(function (){
    Route::post('/', [RatingController::class, 'store']); // Valorar historia
    Route::get('/{id_story}/average', [RatingController::class, 'getAverageRating']); // Promedio de
    calificaciones
});
```

Interconexión:

- Laravel almacena likes y calificaciones en la base de datos.
- React usa botones para que los usuarios den like o valoren historias.

Seguidores (Followers)

```
Route::prefix('followers')->middleware(['auth:sanctum', 'token.expiration'])->group(function (){
    Route::post('/{user}', [FollowerController::class, 'followUser']);
```



```
Route::delete('/{user}', [FollowerController::class, 'unFollowUser']);  
  
});
```

Interconexión:

- Laravel mantiene una tabla de seguidores para manejar relaciones entre usuarios.
- React muestra los seguidores y permite seguir/dejar de seguir con botones.

Desarrollo y estructura del código**Principales clases y funciones*****User.php (Usuario)***

Representa a los usuarios de la plataforma.

Funciones principales:

- `stories()`: Relación uno a muchos con Story (Un usuario puede escribir muchas historias).
- `comments()`: Relación uno a muchos con Comment (Un usuario puede hacer muchos comentarios).
- `likes()`: Relación con Like (Un usuario puede dar like a muchas historias).
- `role()`: Relación con Role (Cada usuario tiene un rol).

Story.php (Historia)

Representa las historias publicadas por los usuarios.

Funciones principales:

- `author()`: Relación muchos a uno con User (Cada historia tiene un autor).
- `comments()`: Relación uno a muchos con Comment (Una historia puede tener varios comentarios).
- `likes()`: Relación uno a muchos con Like (Una historia puede recibir muchos likes).



- ratings(): Relación uno a muchos con Rating (Usuarios pueden valorar la historia).
- category(): Relación muchos a uno con Category (Una historia pertenece a una categoría).

Comment.php (Comentario)

Representa los comentarios de los usuarios en las historias.

Funciones principales:

- user(): Relación muchos a uno con User (Cada comentario tiene un autor).
- story(): Relación muchos a uno con Story (Cada comentario pertenece a una historia).

Like.php (Me gusta)

Registra los "me gusta" que los usuarios dan a las historias.

Funciones principales:

- user(): Relación muchos a uno con User (Cada like pertenece a un usuario).
- story(): Relación muchos a uno con Story (Cada like pertenece a una historia).

Rating.php (Calificación)

Almacena las valoraciones que los usuarios dan a las historias.

Funciones principales:

- user(): Relación muchos a uno con User (Cada calificación pertenece a un usuario).
- story(): Relación muchos a uno con Story (Cada calificación pertenece a una historia).

Category.php (Categoría)

Clasifica las historias en diferentes categorías.

Funciones principales:

- stories(): Relación uno a muchos con Story (Una categoría tiene varias historias).

Role.php (Rol de usuario)

Define los roles de usuario en la plataforma (Ejemplo: escritor, lector, admin).

Funciones principales:

- users(): Relación uno a muchos con User (Un rol puede pertenecer a varios usuarios).

Subscription.php (Suscripción)

Maneja las suscripciones pagas de los usuarios.

Funciones principales:

- user(): Relación muchos a uno con User (Cada suscripción pertenece a un usuario).

Invoice.php (Factura)

Maneja las facturas generadas para las suscripciones.

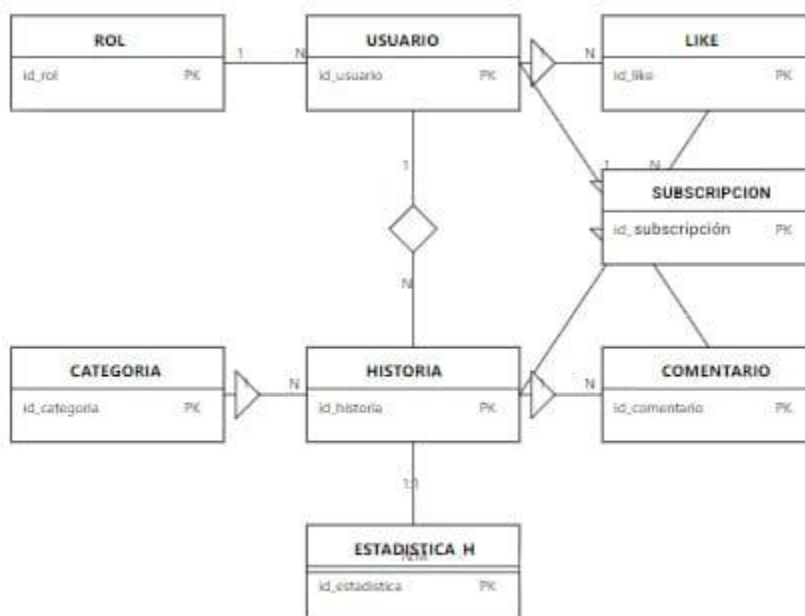
Funciones principales:

- user(): Relación muchos a uno con User (Cada factura pertenece a un usuario).

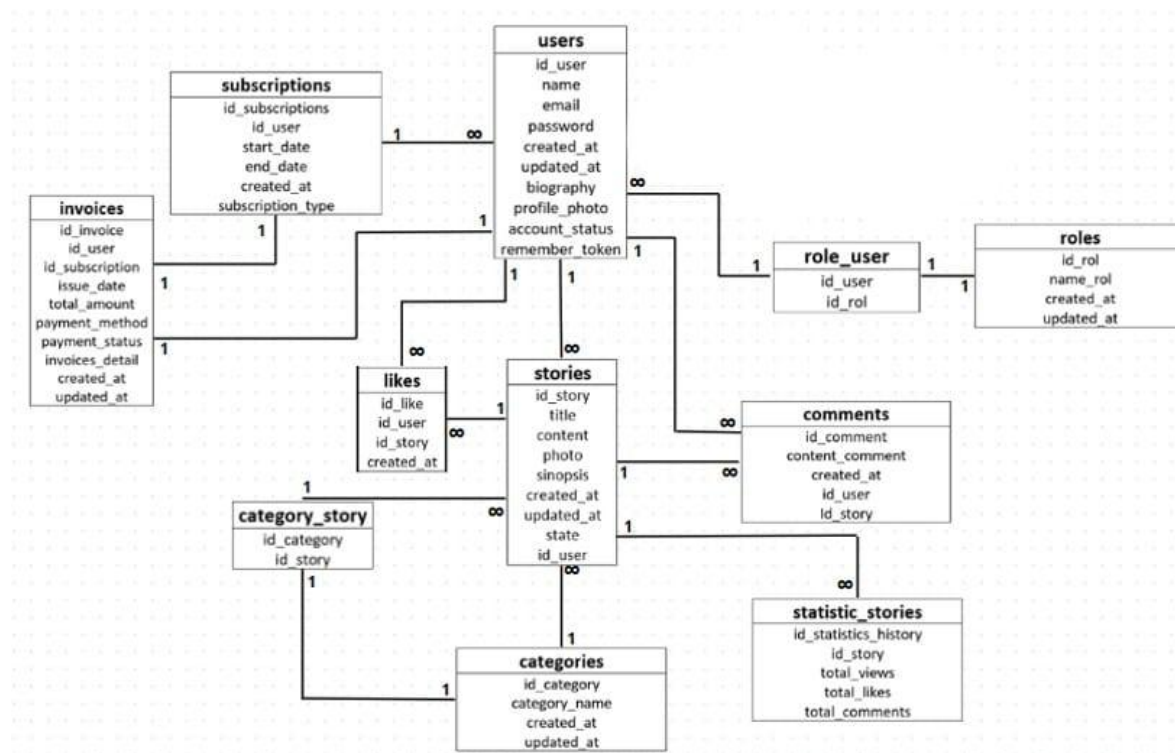
Base de datos

Modelo de datos y diagramas entidad-relación

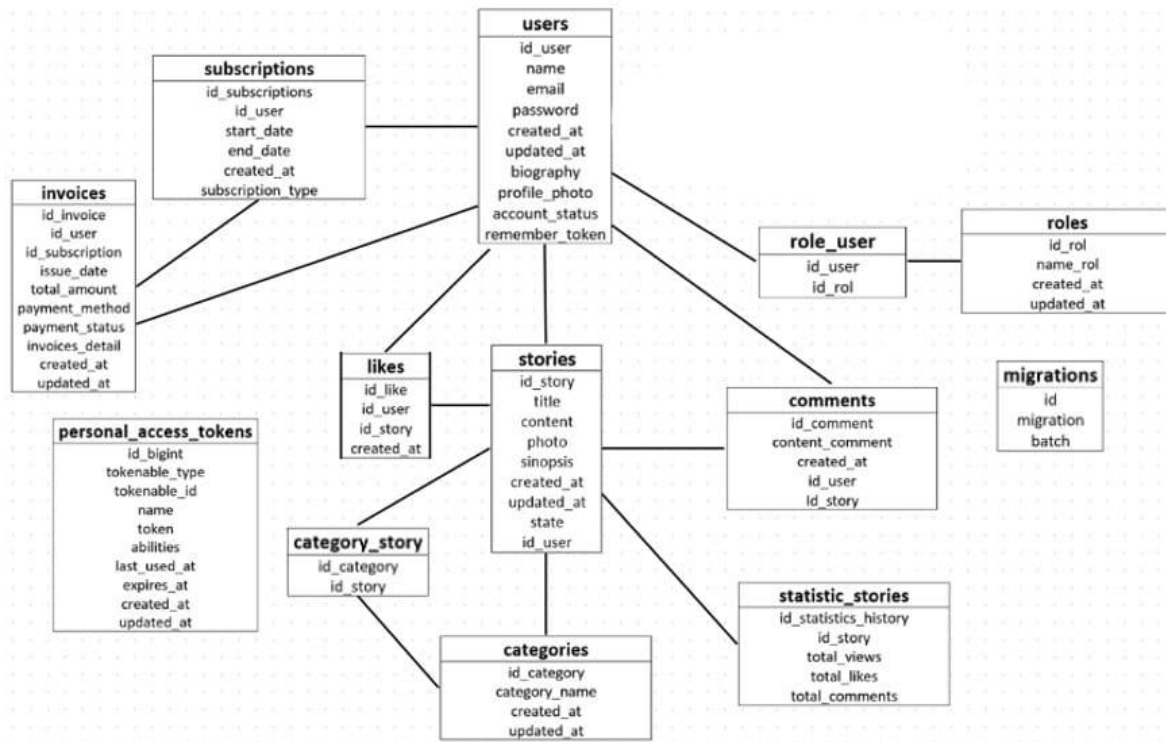
Modelo de datos



Modelo entidad - relación



Tablas principales y relaciones



Procedimientos almacenados y consultas SQL relevantes

-- Crear tabla de usuarios

```

CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    role ENUM('user', 'admin') DEFAULT 'user',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP
);
  
```

-- Procedimiento para registrar un usuario

DELIMITER //

CREATE PROCEDURE RegisterUser(IN p_name VARCHAR(255), IN p_email

VARCHAR(255), IN p_password VARCHAR(255))

BEGIN

INSERT INTO users (name, email, password) VALUES (p_name, p_email, p_password);

END //

DELIMITER ;

-- Procedimiento para obtener todos los usuarios

DELIMITER //

CREATE PROCEDURE GetAllUsers()

BEGIN

SELECT id, name, email, role FROM users;

END //

DELIMITER ;

-- Procedimiento para eliminar un usuario

DELIMITER //

CREATE PROCEDURE DeleteUser(IN p_id INT)

BEGIN

DELETE FROM users WHERE id = p_id;



END //

DELIMITER ;

-- Crear tabla de historias

CREATE TABLE stories (

id INT AUTO_INCREMENT PRIMARY KEY,

user_id INT,

title VARCHAR(255) NOT NULL,

content TEXT NOT NULL,

status ENUM('public', 'private') DEFAULT 'public',

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE

CURRENT_TIMESTAMP,

FOREIGN KEY (user_id) REFERENCES users(id)

);

-- Procedimiento para obtener todas las historias

DELIMITER //

CREATE PROCEDURE GetAllStories()

BEGIN

SELECT s.id, s.title, s.content, u.name AS author, s.status

FROM stories s

JOIN users u ON s.user_id = u.id;



```
END //
```

```
DELIMITER ;
```

```
-- Procedimiento para crear una historia
```

```
DELIMITER //
```

```
CREATE PROCEDURE CreateStory(IN p_user_id INT, IN p_title VARCHAR(255), IN  
p_content TEXT)
```

```
BEGIN
```

```
    INSERT INTO stories (user_id, title, content) VALUES (p_user_id, p_title, p_content);
```

```
END //
```

```
DELIMITER ;
```

```
-- Crear tabla de comentarios
```

```
CREATE TABLE comments (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    story_id INT,
```

```
    user_id INT,
```

```
    comment TEXT NOT NULL,
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    FOREIGN KEY (story_id) REFERENCES stories(id),
```

```
    FOREIGN KEY (user_id) REFERENCES users(id)
```

```
);
```



-- Procedimiento para obtener comentarios de una historia

DELIMITER //

CREATE PROCEDURE GetCommentsByStory(IN p_story_id INT)

BEGIN

SELECT c.id, c.comment, u.name AS author, c.created_at

FROM comments c

JOIN users u ON c.user_id = u.id

WHERE c.story_id = p_story_id;

END //

DELIMITER ;

-- Procedimiento para agregar un comentario a una historia

DELIMITER //

CREATE PROCEDURE AddComment(IN p_story_id INT, IN p_user_id INT, IN p_comment
TEXT)

BEGIN

INSERT INTO comments (story_id, user_id, comment) VALUES (p_story_id, p_user_id,
p_comment);

END //

DELIMITER ;

API y servicios

Documentación de endpoints (REST, GraphQL, SOAP, etc.)

Rutas para autenticación

```
Route::controller(AuthController::class)->group(function () {

    Route::post('/register', 'register'); // Registrar nuevo usuario

    Route::post('/login', 'login'); // Loguear usuario

    Route::middleware(['auth:sanctum', 'token.expiration']->post('/logout', 'logout'); // Cerrar
sesión

});
```

Rutas para usuarios

```
Route::prefix('users')->middleware(['auth:sanctum', 'token.expiration']->controller(UserController::class)->group(function () {

    Route::get('/', 'index'); // Obtener todos los usuarios

    Route::get('{id}', 'show'); // Obtener todos los usuarios

    Route::delete('{id}', 'deleteAccount'); // Eliminar cuenta

    Route::put('{id}/role', 'updateRole'); // Actualizar rol

    Route::get('{user}/stories', [StoryController::class, 'getUserStories']);
```

Historias de un usuario

```
Route::get('{user}/comments', [CommentController::class, 'getUserComments']); //
Comentarios de un usuario
```

```
});
```

```
Route::middleware(['auth:sanctum', 'token.expiration'])->group(function () {
    Route::get('profile', [ProfileController::class, 'show']); // Obtener perfil
    Route::put('profile/{user}', [ProfileController::class, 'update']);

    Route::put('profile/{user}/role', [ProfileController::class, 'updateRole']);
    Route::put('profile/{user}/status', [ProfileController::class, 'updateStatus']);
});
```

Resetear contraseña

```
Route::post('/forgot-password', [PasswordResetController::class, 'sendResetLink']);
Route::post('/reset-password', [PasswordResetController::class, 'resetPassword']);
```

Rutas para historias

```
Route::prefix('stories')->group(function () {
```

Rutas públicas

```
    Route::get('/', [StoryController::class, 'index']); // Obtener todas las historias
    Route::get('/categories', [CategoryController::class, 'index']); // Obtener historias por categoría
    Route::get('/{id}', [StoryController::class, 'show']); // Obtener datos
    Route::get('/{story}/comments', [CommentController::class, 'index']); // Obtener comentarios
    de una historia
```



```
Route::middleware(['auth:sanctum', 'token.expiration'])->group(function () {
```

Rutas protegidas

```
Route::post('/', [StoryController::class, 'store']); // Crear nueva historia
```

```
Route::put('/{story}', [StoryController::class, 'update']); // Actualizar
```

```
Route::delete('/{story}', [StoryController::class, 'destroy']); // Eliminar
```

```
Route::get('{id}/owner', [StoryController::class, 'getStoryOwner']); // Obtener dueño de una historia
```

```
Route::post('/{story}/comments', [CommentController::class, 'store']); // Crear comentario
});
});
```

Actualizar estado de una historia

```
Route::middleware(['auth:sanctum', 'token.expiration'])->group(function () {
Route::put('/stories/{story}/status', [StoryController::class, 'updateStatus']);
});
```

Rutas para los comentarios

```
Route::prefix('comments')->group(function(){
```

Rutas públicas

```
Route::get('/{comment}', [CommentController::class, 'show']); //Obtener un comentario específico
```




```
Route::get('/{storyId}/comment', [CommentController::class, 'index']);
```

```
Route::middleware(['auth:sanctum', 'token.expiration'])->group(function(){
```

Rutas protegidas

```
Route::post('/{storyId}', [CommentController::class, 'store']);
```

```
Route::put('/{comment}', [CommentController::class, 'update']); //Actualizar un comentario
```

```
Route::delete('/{comment}', [CommentController::class, 'destroy']); //Eliminar un  
comentario
```

```
Route::get('/{id}/owner', [CommentController::class, 'getCommentOwner']); //Obtener  
dueño de un comentario
```

```
});
```

```
});
```

Rutas para interacciones

```
Route::prefix('interactions')->middleware(['auth:sanctum', 'token.expiration'])->group(function ()  
{
```

```
Route::put('/{story}', [LikeController::class, 'toggleInteraction']); // Alternar interacciones
```

```
Route::get('/{story}/count', [LikeController::class, 'getStoryInteractions']); // Contar  
interacciones
```

```
});
```

Rutas para ratings

```
Route::prefix('ratings')->middleware(['auth:sanctum', 'token.expiration'])->group(function () {  
    Route::post('/', [RatingController::class, 'store']);  
    Route::get('/{id_story}/user', [RatingController::class, 'getUserRating']);  
    Route::get('/{id_story}/average', [RatingController::class, 'getAverageRating']);  
    Route::delete('/{id_story}', [RatingController::class, 'destroy']);  
});
```

Rutas para suscripciones

```
Route::prefix('subscriptions')->middleware(['auth:sanctum', 'token.expiration'])->group(function  
() {  
    Route::post('/', [SubscriptionController::class, 'createSubscription']);  
    Route::get('/', [SubscriptionController::class, 'getUserSubscriptions']);  
    Route::delete('/{subscriptionId}', [SubscriptionController::class, 'cancelSubscription']);  
});
```

Rutas para facturas

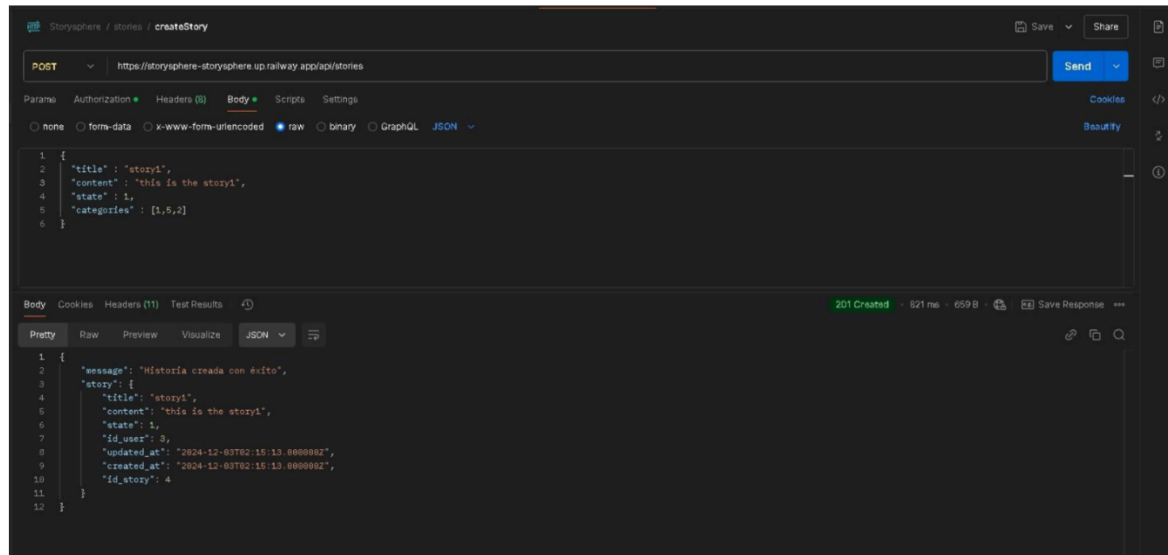
```
Route::prefix('invoices')->middleware(['auth:sanctum', 'token.expiration'])->group(function () {  
    Route::get('/', [InvoiceController::class, 'index']);  
    Route::get('/{id}', [InvoiceController::class, 'show']);  
    Route::post('/', [InvoiceController::class, 'store']);  
    Route::put('/{id}/status', [InvoiceController::class, 'updateStatus']);  
});
```

```
Route::delete('/{id}', [InvoiceController::class, 'destroy']);

});
```

Ejemplos de solicitudes y respuestas

create story



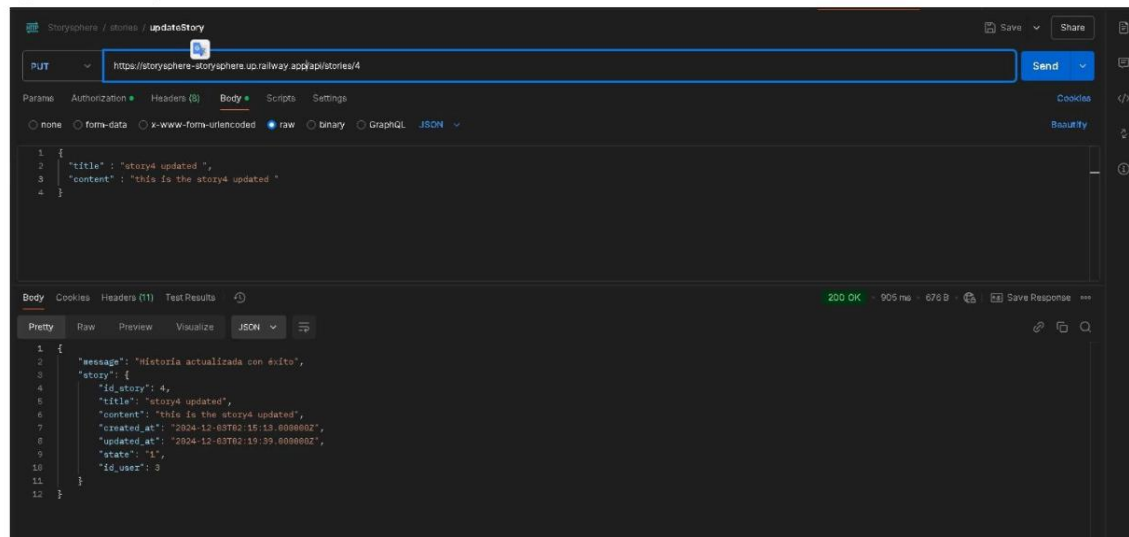
The screenshot shows a REST client interface for a POST request to `https://storysphere-storysphere.up.railway.app/api/stories`. The request body is a JSON object with the following structure:

```
1 {
2   "title": "story1",
3   "content": "this is the story1",
4   "state": 1,
5   "categories": [1,5,2]
6 }
```

The response is a 201 Created status with the following JSON body:

```
1 {
2   "message": "Historia creada con éxito",
3   "story": {
4     "title": "story1",
5     "content": "this is the story1",
6     "state": 1,
7     "id_user": 3,
8     "updated_at": "2024-12-03T02:15:13.000000Z",
9     "created_at": "2024-12-03T02:15:13.000000Z",
10    "id_story": 4
11  }
12 }
```

updateStory



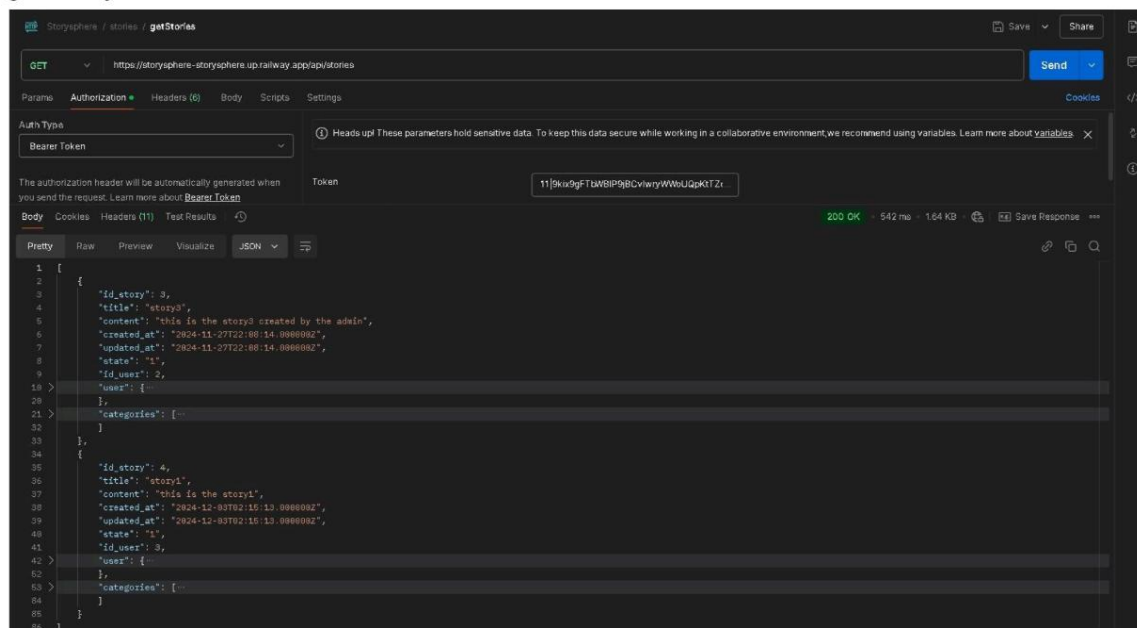
The screenshot shows a REST client interface for a PUT request to `https://storysphere-storysphere.up.railway.app/api/stories/4`. The request body is a JSON object with the following structure:

```
1 {
2   "title": "story4 updated ",
3   "content": "this is the story4 updated "
4 }
```

The response is a 200 OK status with the following JSON body:

```
1 {
2   "message": "Historia actualizada con éxito",
3   "story": {
4     "id_story": 4,
5     "title": "story4 updated",
6     "content": "this is the story4 updated",
7     "created_at": "2024-12-03T02:15:13.000000Z",
8     "updated_at": "2024-12-03T02:19:39.000000Z",
9     "state": "1",
10    "id_user": 3
11  }
12 }
```

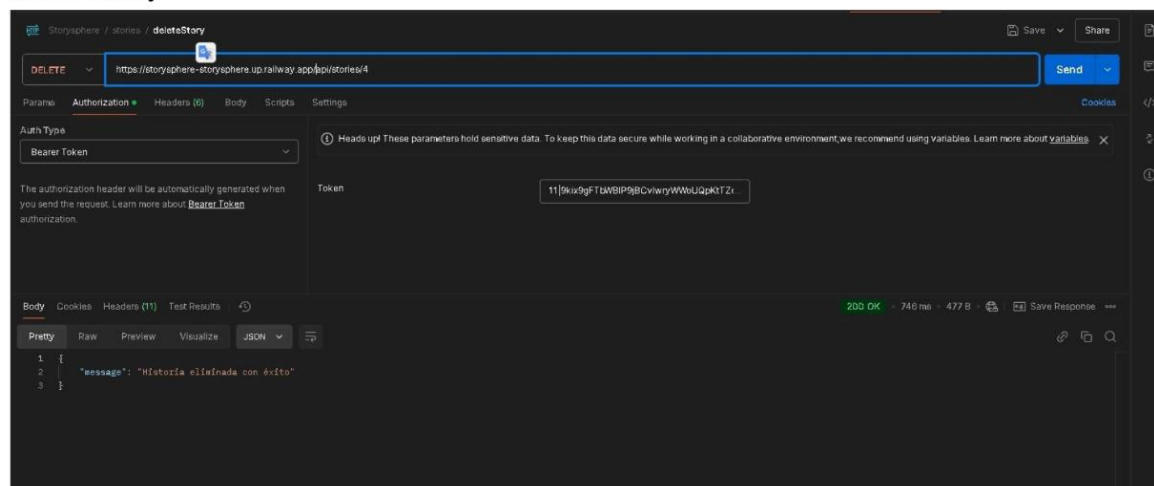
getStory



The screenshot shows a REST client interface for a StorySphere API. The URL is `https://storysphere-storysphere.up.railway.app/api/stories`. The request is a GET method. The response is a 200 OK status, indicating a successful retrieval of stories. The response body is a JSON array containing two story objects.

```
1 {
2   {
3     "id_story": 3,
4     "title": "story3",
5     "content": "this is the story3 created by the admin",
6     "created_at": "2024-11-27T22:08:14.090000Z",
7     "updated_at": "2024-11-27T22:08:14.090000Z",
8     "state": "1",
9     "id_user": 2,
10    "users": [
11      {
12        "id": 2,
13        "username": "admin",
14        "password": "123456",
15        "email": "admin@storysphere.com",
16        "created_at": "2024-11-27T22:08:14.090000Z",
17        "updated_at": "2024-11-27T22:08:14.090000Z",
18        "state": "1",
19        "id_role": 1,
20        "roles": [
21          {
22            "id_role": 1,
23            "name": "admin",
24            "description": "Administrador del sistema",
25            "created_at": "2024-11-27T22:08:14.090000Z",
26            "updated_at": "2024-11-27T22:08:14.090000Z",
27            "state": "1",
28            "id_user": 2,
29            "users": [
30              {
31                "id": 2,
32                "username": "admin",
33                "password": "123456",
34                "email": "admin@storysphere.com",
35                "created_at": "2024-11-27T22:08:14.090000Z",
36                "updated_at": "2024-11-27T22:08:14.090000Z",
37                "state": "1",
38                "id_role": 1,
39                "roles": [
40                  {
41                    "id_role": 1,
42                    "name": "admin",
43                    "description": "Administrador del sistema",
44                    "created_at": "2024-11-27T22:08:14.090000Z",
45                    "updated_at": "2024-11-27T22:08:14.090000Z",
46                    "state": "1",
47                    "id_user": 2,
48                    "users": [
49                      {
50                        "id": 2,
51                        "username": "admin",
52                        "password": "123456",
53                        "email": "admin@storysphere.com",
54                        "created_at": "2024-11-27T22:08:14.090000Z",
55                        "updated_at": "2024-11-27T22:08:14.090000Z",
56                        "state": "1",
57                        "id_role": 1,
58                        "roles": [
59                          {
60                            "id_role": 1,
61                            "name": "admin",
62                            "description": "Administrador del sistema",
63                            "created_at": "2024-11-27T22:08:14.090000Z",
64                            "updated_at": "2024-11-27T22:08:14.090000Z",
65                            "state": "1",
66                            "id_user": 2,
67                            "users": [
68                              {
69                                "id": 2,
70                                "username": "admin",
71                                "password": "123456",
72                                "email": "admin@storysphere.com",
73                                "created_at": "2024-11-27T22:08:14.090000Z",
74                                "updated_at": "2024-11-27T22:08:14.090000Z",
75                                "state": "1",
76                                "id_role": 1,
77                                "roles": [
78                                  {
79                                    "id_role": 1,
80                                    "name": "admin",
81                                    "description": "Administrador del sistema",
82                                    "created_at": "2024-11-27T22:08:14.090000Z",
83                                    "updated_at": "2024-11-27T22:08:14.090000Z",
84                                    "state": "1",
85                                    "id_user": 2,
86                                    "users": [
87                                    ]
88                                  ]
89                                ]
90                              ]
91                            ]
92                          ]
93                        ]
94                      ]
95                    ]
96                  ]
97                ]
98              ]
99            ]
100           ]
101         ]
102       ]
103     ]
104   }
105 }
```

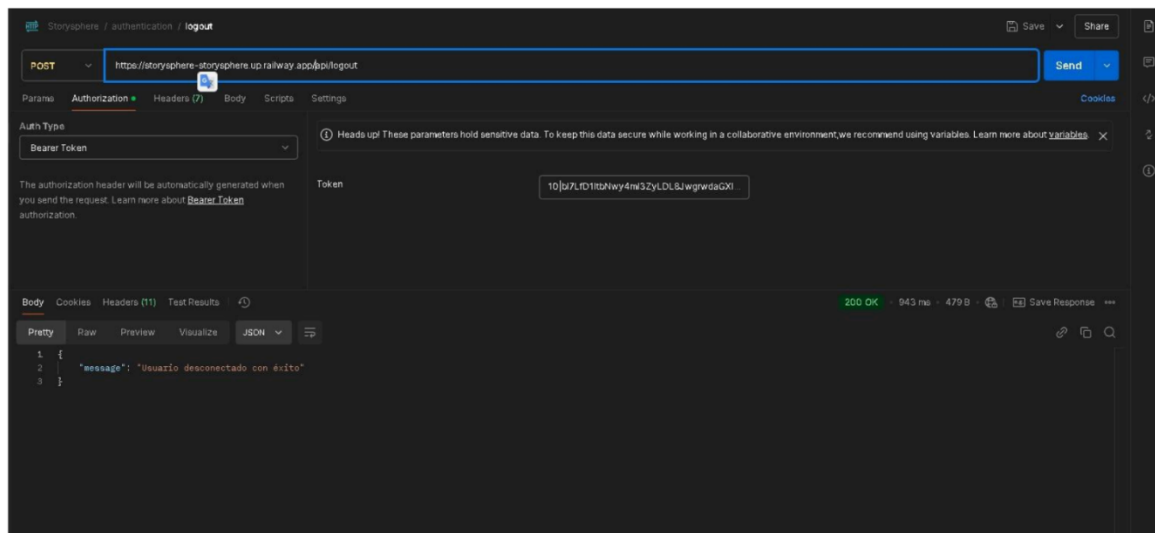
deleteStory



The screenshot shows a REST client interface for a StorySphere API. The URL is `https://storysphere-storysphere.up.railway.app/api/stories/4`. The request is a DELETE method. The response is a 200 OK status, indicating a successful deletion of the story. The response body is a JSON object with a message.

```
1 {
2   "message": "Historia eliminada con éxito"
3 }
```

user

A screenshot of a REST client interface, likely Postman, showing a POST request to the endpoint `https://storysphere-storysphere.up.railway.app/api/logout`. The request is configured with the "Authorization" tab selected, showing a Bearer Token. The response is displayed in the "Body" tab, showing a JSON object with a success message.

StorySphere / authentication / logout

POST `https://storysphere-storysphere.up.railway.app/api/logout` Send

Params Authorization Headers (7) Body Scripts Settings Cookies

Auth Type: Bearer Token

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

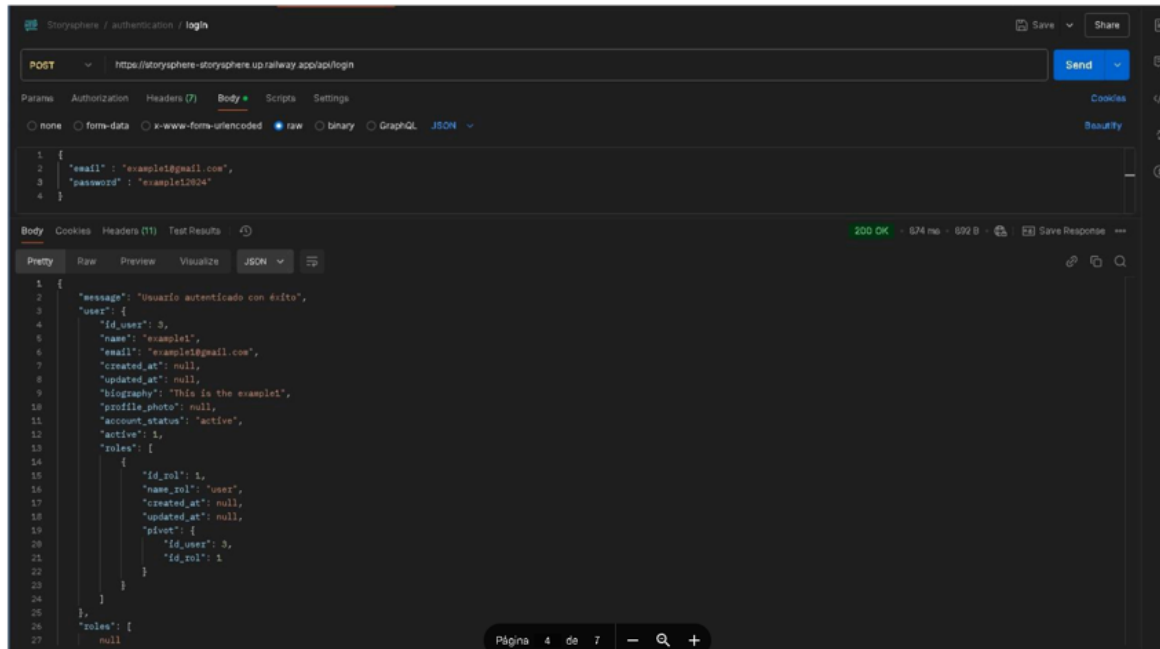
Token: `10jz7LID1ttNwy4m3ZyLDL8JwgrwdaGXi`

Body Cookies Headers (11) Test Results 200 OK · 943 ms · 479 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "Usuario desconectado con éxito"
3 }
```

Login



Storysphere / authentication / login

POST <https://storysphere-storysphere.up.railway.app/api/login> Send

Params Authorization Headers (7) **Body** Scripts Settings Cookies Beauty

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON

```

1 {
2   "email": "example@gmail.com",
3   "password": "example1234"
4 }

```

Body Cookies Headers (11) Test Results 200 OK 674 ms 692 B Save Response

Pretty Raw Preview Visualize **JSON** +

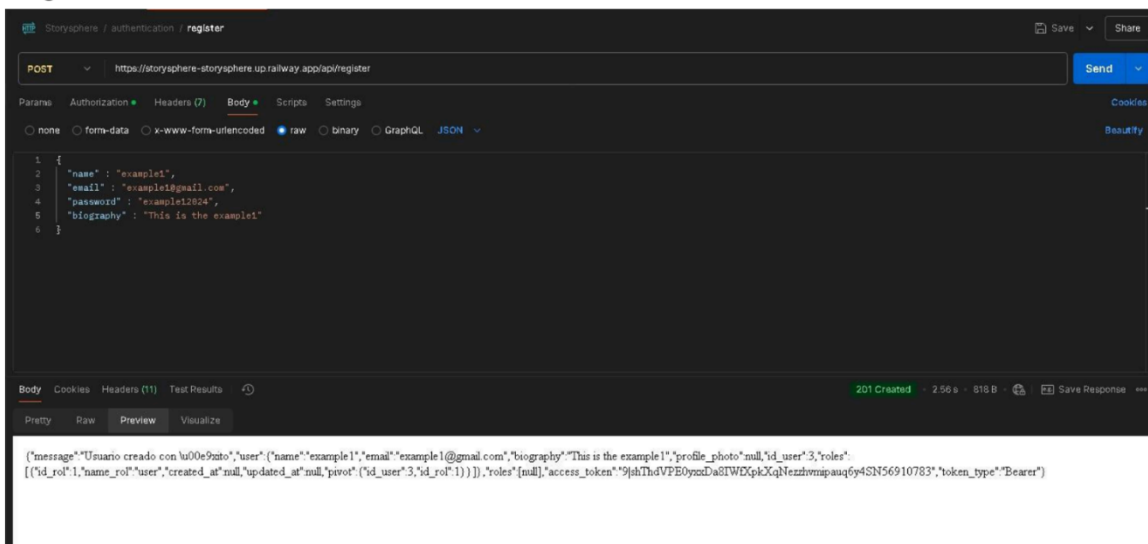
```

1 {
2   "message": "Usuario autenticado con éxito",
3   "user": {
4     "id_user": 3,
5     "name": "example1",
6     "email": "example@gmail.com",
7     "created_at": null,
8     "updated_at": null,
9     "biography": "This is the example1",
10    "profile_photo": null,
11    "account_status": "active",
12    "active": 1,
13    "roles": [
14      {
15        "id_role": 1,
16        "name_role": "user",
17        "created_at": null,
18        "updated_at": null,
19        "pivot": {
20          "id_user": 3,
21          "id_role": 1
22        }
23      }
24    ]
25  },
26   "roles": [
27     null

```

Página 4 de 7

Register



Storysphere / authentication / register

POST <https://storysphere-storysphere.up.railway.app/api/register> Send

Params Authorization Headers (7) **Body** Scripts Settings Cookies Beauty

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON

```

1 {
2   "name": "example1",
3   "email": "example@gmail.com",
4   "password": "example1234",
5   "biography": "This is the example1"
6 }

```

Body Cookies Headers (11) Test Results 201 Created 2.56 s 816 B Save Response

Pretty Raw **Preview** Visualize

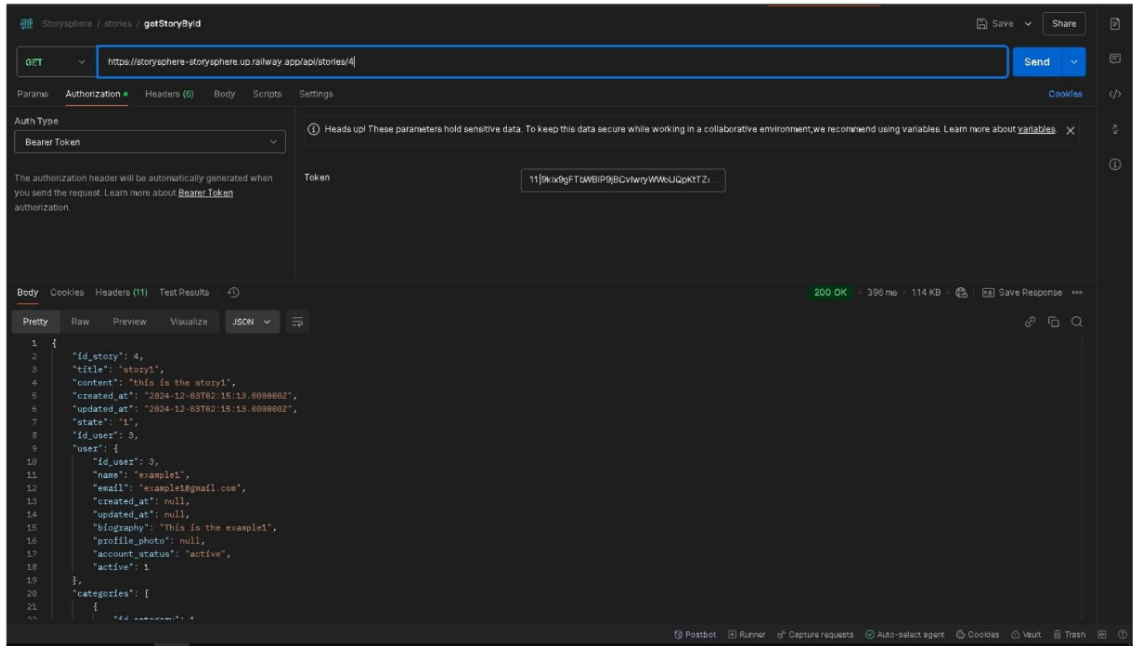
```

{"message":"Usuario creado con éxito","user":{"name":"example1","email":"example@gmail.com","biography":"This is the example1","profile_photo":null,"id_user":3,"roles":[{"id_role":1,"name_role":"user","created_at":null,"updated_at":null,"pivot":{"id_user":3,"id_role":1}}],"roles":[null],"access_token":"9jshThdVFEoyozDa8tWEXpkXqNzzhvmipauq6y4SN56910783","token_type":"Bearer"}

```



getByIdStory



Diccionario de variables

Users

Atributo	Tipo de dato	Tamaño	Llave primaria	Llave foránea	Obligatorio	Null	Auto incrementable	Única
id_user	BigIncrements	20	X		X		X	
name	VARCHAR	100			X			
email	VARCHAR	150			X			X
password	TEXT				X			
created_at	TIMESTAMP				X	SI		
updated_at	TIMESTAMP				X	SI		

biography	TEXT				X	SI		
profile_photo	VARC HAR	150			X	SI		
account_status	VARC HAR	100			X	SI		
remember_token	VARC HAR	188			X	SI		

Stories

Atributo	Tipo de dato	Tamaño	Llave primaria	Llave foránea	Obligatorio	Null	Auto incrementable	Única
id_story	BigIncrements		X		X	X	X	
title	String	255		X	X	X		
content	Text	150			X	X		X
photo	Varchar	248			X	X		
sinopsis	Text				X	X		
created_at	Timestamp				X	X		
updated_at	Timestamp				X	X		
state	String	50			X			
id_user	Bigint			X	X	X		

Subscriptions

Atributo	Tipo de dato	Tamaño	Llave primaria	Llave foránea	Obligatorio	Null	Auto incrementable	Única
id_subscriptions	BigIncrements		X		X		X	
id_user	Bigint	255		X	X			
start_date	Date	150			X			
end_date	Date	248				X		

e								
created_at	Timestamp				X			
updated_at	Timestamp				X			
subscription_type	Enum	50			X			
mercado_pago_subscription_id	Varchar			X	X	X		
mercado_pago_plan_id	Varchar					X		
mercado_pago_status	Enum					X		
mercado_pago_created_at	Timestamp					X		
mercado_pago_updated_at	Timestamp					X		
mercado_pago_response	Timestamp					X		

Statistic_stories

Atributo	Tipo de dato	Tamaño	Llave primaria	Llave foránea	Obligatorio	Null	Auto incrementable	Única
----------	--------------	--------	----------------	---------------	-------------	------	--------------------	-------

id_statistic_history	BigIncrements		X		X		X	
id_story	Bigint	255		X	X			
total_views	Integer	150						
total_likes	Integer	248						
total_comments	Integer							

Roles

Atributo	Tipo de dato	Tamaño	Llave primaria	Llave foránea	Obligatorio	Null	Auto incrementable	Única
id_rol	BigIncrements		X		X		X	
name_rol	String	100			X			
created_at	Timestamp				X			
updated_at	Timestamp				X			

Likes

Atributo	Tipo de dato	Tamaño	Llave primaria	Llave foránea	Obligatorio	Null	Auto incrementable	Única
id_like	BigIncrements		X		X		X	

id_user	Bigint			X	X			
id_story	Bigint			X	X			
created_at	Timestamp				X			

Invoices

Atributo	Tipo de dato	Tamaño	Llave primaria	Llave foránea	Obligatorio	Null	Auto incrementable	Única
id_invoice	BigIncrements		X		X		X	
id_user	Bigint			X	X			
id_subscription	Bigint			X	X			
issue_date	Date				X			
total_amount	Decimal	8,2			X			
payment_method	Enum				X			
payment_status	Enum				X			
invoice_detail	Text				X			
mercado_pago_payment_id	String					X		
mercado_pago_status	String					X		
mercado	String					X		

_pago_p referenc e_id								
mercado _pago_c reated_a t	TimesT amp					X		
mercado _pago_u pdated_ at	Timest amp					X		
mercado _pago_r esponse	Text					X		
created_ at	TimesT amp				X			
updated _at	TimesT amp				X			

Comments

Atributo	Tipo de dato	Tamaño	Llave primaria	Llave foránea	Obligatorio	Null	Auto incrementable	Única
id_com ment	BigIncr ements		X		X		X	
content_ commen t	Text				X			
created_ at	TimesT amp				X			
id_user	Bigint			X	X			
id_story	Bigint			X				

Categories

Atributo	Tipo de dato	Tamaño	Llave primaria	Llave foránea	Obligatorio	Null	Auto incrementable	Única
id_category	BigIncrements		X		X		X	
category_name	String	100		X	X			
created_at	Timestamp			X	X			
updated_at	Timestamp				X			

Category_story

Atributo	Tipo de dato	Tamaño	Llave primaria	Llave foránea	Obligatorio	Null	Auto incrementable	Única
id_category	Bigint		X	X	X		X	
category_name	Bigint		X	X	X			

Role_user

Atributo	Tipo de dato	Tamaño	Llave primaria	Llave foránea	Obligatorio	Null	Auto incrementable	Única
id_user	Bigint		X	X	X			
id_rol	Bigint		X	X	X			

Ratings

Atributo	Tipo de dato	Tamaño	Llave primaria	Llave foránea	Obligatorio	Null	Auto incrementable	Única
id_rating	BigIncrements		X		X		X	
rating	TinyInteger				X			
created_at	Timestamp				X			
updated_at	Timestamp				X			
id_user	Bigint			X	X			
id_story	Bigint			X	X			

Personal_access_tokens

Atributo	Tipo de dato	Tamaño	Llave primaria	Llave foránea	Obligatorio	Null	Auto incrementable	Única
id	BigIncrements		X		X		X	
tokenable_id	Bigint				X			
tokenable_type	String				X			X
tame	String				X			
token	String	64			X			
abilities	Text					X		
last_used_at	Timestamp					X		

expires_ at	TimesT amp					X		
created_ at	TimesT amp				X			
updated_ _at	TimesT amp				X			

Webgrafía

Código fuente

<https://github.com/yey10/storysphere>

Diagramas

<https://docs.google.com/document/d/1iwcntjE7IFCbHVmFEYtLLGJyuM9Y3txELb1TJgAqpk/edit?usp=sharing>